JavaScript 基础概念精讲

1. 概述 (Overview)

1.1 概念定义

JavaScript（简称 JS）是一种具有函数优先特性的轻量级、解释型或即时编译型编程语言。它是 Web 开发的三大基石之一（HTML、CSS、JS），也是目前世界上最流行的编程语言。尽管它以作为 Web 页面的脚本语言而闻名，但非浏览器环境（如 Node.js）也广泛使用它。JavaScript 是一种基于原型编程、多范式的动态脚本语言，并且支持面向对象、命令式和声明式（如函数式编程）风格。

1.2 核心价值

JavaScript 的核心价值在于其**全栈能力**和**跨平台特性**。

- **交互性**：它赋予了网页动态交互的能力，从简单的动画到复杂的单页应用（SPA）。
- **全栈开发**：通过 Node.js，JavaScript 可以在服务器端运行，实现前后端语言统一。
- **生态系统**：拥有全球最大的开源包管理器 npm，提供了数百万个复用包。

1.3 发展历程

- **1995 年**：Brendan Eich 在 Netscape 用 10 天时间设计了 JavaScript 的原型。
- **1997 年**：ECMA 发布 ECMA-262 标准，正式确立 ECMAScript（ES）。
- **2009 年**：ES5 发布，带来了严格模式（strict mode）、JSON 支持等。
- **2015 年**：ES6 (ES2015) 发布，这是史上最大的版本更新，引入了 class、module、arrow function、promise、let/const 等，标志着现代 JavaScript 的诞生。
- **2016 年至今**：每年发布一个新版本（ES2016, ES2017...），持续引入 async/await、Rest/Spread 等特性。

---

2. 核心概念与原理 (Concepts & Principles)

2.1 变量与数据类型 (Variables & Types)

JavaScript 是一种弱类型（动态类型）语言，这意味着变量不需要声明类型，且类型可以在运行时改变。

2.1.1 变量声明

ES6 引入了 `let` 和 `const`，加上传统的 `var`，共有三种声明方式。

- **var**: 函数作用域，存在变量提升（Hoisting），可重复声明。
- **let**: 块级作用域，不存在变量提升（暂时性死区 TDZ），不可重复声明。
- **const**: 块级作用域，声明常量，必须初始化，引用不可变（但对象属性可变）。

```
// 示例：作用域与提升
console.log(a); // undefined (提升)
var a = 1;

// console.log(b); // ReferenceError: Cannot access 'b' before initialization
(TDZ)
let b = 2;

const USER = { name: "Alice" };
USER.name = "Bob"; // 合法：修改对象属性
// USER = {}; // TypeError: Assignment to constant variable
```

2.1.2 数据类型

JavaScript 数据类型分为**基本数据类型（Primitive）**和**引用数据类型（Reference）**。

基本数据类型（栈内存存储，不可变）：

1. **String**: 文本数据。
2. **Number**: 整数和浮点数（基于 IEEE 754 标准）。
3. **Boolean**: true / false。
4. **Null**: 空值，表示"无对象"。
5. **Undefined**: 未定义，表示变量已声明但未赋值。
6. **Symbol** (ES6): 唯一标识符。
7. **BigInt** (ES2020): 任意精度的整数。

引用数据类型（堆内存存储，可变）：

1. **Object**: 普通对象、数组、函数、日期等。

```
// 示例：BigInt 与 Symbol
const bigNum = 9007199254740991n; // 必须加 n 后缀
const sym1 = Symbol("id");
const sym2 = Symbol("id");
console.log(sym1 === sym2); // false，Symbol 保证唯一性
```

2.1.3 类型转换

- **显式转换**: `Number()`, `String()`, `Boolean()`
- **隐式转换**: `+` 运算符、`==` 比较等。

2.1.4 深入 V8 引擎与内存管理 (Deep Dive: V8 Engine)

理解 JavaScript 的内存管理机制对于编写高性能代码至关重要。V8 是 Chrome 和 Node.js 使用的 JavaScript 引擎。

1. 内存模型 (Memory Model)

- **栈内存 (Stack)**: 存储基本数据类型和执行上下文。内存连续，分配和释放速度极快。
- **堆内存 (Heap)**: 存储引用数据类型（对象）。内存不连续，需要垃圾回收器（GC）管理。

2. 垃圾回收机制 (Garbage Collection)
V8 使用分代回收策略，将堆内存分为**新生代 (New Space)** 和 **老生代 (Old Space)**。

- **新生代 (Young Generation)**:
  - 存储生命周期短的对象。

    - 使用 **Scavenge 算法**（Cheney 算法实现）。
    - 将内存分为 `From` 空间和 `To` 空间。GC 时，将存活对象从 From 复制到 To，然后交换空间。
    - 速度快，但空间利用率低（只有一半可用）。
- **老生代 (Old Generation)**:
    - 存储生命周期长或从新生代晋升的对象。
    - 使用 **标记-清除 (Mark-Sweep)** 和 **标记-整理 (Mark-Compact)** 算法。
    - **Mark-Sweep**: 标记存活对象，清除未标记对象（产生内存碎片）。
    - **Mark-Compact**: 在清除前将存活对象向一端移动，解决碎片问题。

3. V8 编译管线

- **Parser**: 将 JS 源码转换为抽象语法树 (AST)。
- **Ignition (解释器)**: 将 AST 转换为字节码并执行。
- **TurboFan (优化编译器)**: 收集运行时的类型信息，将热点代码（Hot Code）编译为优化的机器码。
    - **优化陷阱**: 如果变量类型频繁变化，TurboFan 会执行“去优化 (Deoptimization)”，回退到字节码执行，导致性能下降。

```
// 优化建议：保持对象形状一致 (Hidden Class)
function Point(x, y) {
  this.x = x;
  this.y = y;
}

const p1 = new Point(1, 2);
const p2 = new Point(3, 4);

// 不推荐：动态添加属性会改变 Hidden Class，阻碍优化
p1.z = 5;
```

2.2 执行上下文与作用域 (Execution Context & Scope)

2.2.1 执行上下文 (Execution Context)

JavaScript 代码运行的环境。

- **全局执行上下文**: 默认环境，`window` (浏览器) 或 `global` (Node.js)。
- **函数执行上下文**: 每次调用函数时创建。
- **Eval 执行上下文**: `eval()` 执行的代码（不推荐使用）。

执行上下文包含三个部分：

1. **变量对象 (VO/AO)**: 存储变量、函数声明、参数。
2. **作用域链 (Scope Chain)**: 用于解析变量引用。
3. **this 指向**: 动态绑定。

2.2.2 闭包 (Closure)

闭包是指有权访问另一个函数作用域中变量的函数。它是 JavaScript 最强大的特性之一。

原理: 内部函数引用了外部函数的变量，导致外部函数的执行上下文无法被销毁（垃圾回收机制 GC 不回收）。

```
function createCounter() {
  let count = 0; // 私有变量
  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
// count 变量依然存在于内存中，只能通过 counter 函数访问
```

应用场景:

- 数据私有化（模拟私有方法）。
- 函数柯里化（Currying）。
- 模块化模式。

2.3 原型与原型链 (Prototype & Prototype Chain)

JavaScript 是一门基于原型的语言，而不是基于类（尽管 ES6 引入了 class 语法糖）。

- **Prototype**: 每个函数都有一个 `prototype` 属性，指向原型对象。
- ****proto****: 每个对象都有一个 `__proto__` 属性，指向构造函数的原型。
- **原型链**: 当访问对象的属性时，如果对象本身没有，会沿着 `__proto__` 向上查找，直到 `null`。

```
function Person(name) {
  this.name = name;
}

Person.prototype.sayHello = function () {
  console.log(`Hello, I am ${this.name}`);
};

const p1 = new Person("Tom");
p1.sayHello(); // 自身没有 sayHello，沿原型链找到 Person.prototype.sayHello

console.log(p1.__proto__ === Person.prototype); // true
console.log(Person.prototype.__proto__ === Object.prototype); // true
console.log(Object.prototype.__proto__ === null); // true (链的顶端)
```

2.4 异步编程 (Asynchronous Programming)

JavaScript 是单线程的，通过**事件循环 (Event Loop)** 实现异步非阻塞。

2.4.1 事件循环机制详解 (Event Loop Deep Dive)

理解 Event Loop 是掌握 JS 异步行为的关键。

1. **调用栈 (Call Stack)**: 执行同步代码 (LIFO)。
2. **任务队列 (Task Queue)**:

- **宏任务 (MacroTask)**: `setTimeout`, `setInterval`, `setImmediate` (Node), I/O, UI Rendering, `MessageChannel`.
- **微任务 (MicroTask)**: `Promise.then/catch/finally`, `process.nextTick` (Node), `MutationObserver`, `queueMicrotask`.

执行顺序规则:

1. 执行栈中的同步代码（Script 脚本）。
2. **检查并清空微任务队列**: 依次执行所有微任务。如果在执行微任务过程中产生了新的微任务，会继续追加到队列尾部并在本轮循环中执行完。
3. **UI 渲染**: (浏览器环境) 可能会进行页面重绘。
4. **执行一个宏任务**: 从宏任务队列中取出一个任务执行。
5. 回到步骤 2。

面试题：输出顺序分析

```
console.log("1");

setTimeout(() => {
  console.log("2");
  Promise.resolve().then(() => {
    console.log("3");
  });
}, 0);

new Promise((resolve) => {
  console.log("4");
  resolve();
}).then(() => {
  console.log("5");
});

console.log("6");

// 输出顺序: 1, 4, 6, 5, 2, 3
// 解析:
// 1. 同步: log(1) -> 1
// 2. 宏任务: setTimeout 入队 [macro1]
// 3. 同步: new Promise Executor 立即执行 -> log(4) -> 4
// 4. 微任务: then 回调入队 [micro1]
// 5. 同步: log(6) -> 6
// 6. 清空微任务: 执行 [micro1] -> log(5) -> 5
// 7. 执行宏任务: [macro1] -> log(2) -> 2
// 8. 宏任务中产生微任务: then 入队 [micro2]
// 9. 清空微任务: 执行 [micro2] -> log(3) -> 3
```

2.4.2 Promise

ES6 引入的标准异步解决方案，解决了回调地狱（Callback Hell）。
三种状态: `Pending` (进行中), `Fulfilled` (已成功), `Rejected` (已失败)。

2.4.3 Async/Await

ES2017 引入的语法糖，基于 Promise，使异步代码看起来像同步代码。

```
async function fetchData() {
  try {
    console.log("Start");
    const result = await new Promise((resolve) =>
      setTimeout(() => resolve("Data"), 1000)
    );
    console.log(result); // 1秒后输出 'Data'
  } catch (error) {
    console.error(error);
  }
}
```

---

3. 进阶语言特性 (Advanced Language Features)

3.1 ES6+ 核心特性

3.1.1 箭头函数 (Arrow Function)

- 更简洁的语法。
- **没有自己的** `this`，继承外层作用域的 `this`（词法作用域）。
- 没有 `arguments` 对象。
- 不能作为构造函数（没有 `[[Construct]]` 方法，没有 `prototype`）。

```
const add = (a, b) => a + b;

const obj = {
  value: 10,
  getValue: function () {
    // 普通函数，this 指向调用者 obj
    return () => this.value; // 箭头函数，this 继承自 getValue 的作用域 (obj)
  },
};
```

3.1.2 解构赋值 (Destructuring)

从数组或对象中提取值，对变量进行赋值。

```
// 对象解构
const user = { id: 1, info: { name: "Alice" } };
const {
  info: { name },
  age = 18,
} = user; // 嵌套解构 + 默认值

// 数组解构
const [first, ...rest] = [1, 2, 3, 4]; // first=1, rest=[2,3,4]
```

3.1.3 模块化 (Modules)

- **CommonJS**: Node.js 默认，`require` / `module.exports`，运行时加载，同步加载，输出值的拷贝。
- **ES Modules (ESM)**: 浏览器/现代 Node，`import` / `export`，编译时输出接口，异步加载，输出值的引用。

```
// ESM 示例
// math.js
export const add = (a, b) => a + b;
export default function multiply(a, b) {
  return a * b;
}

// main.js
import multiply, { add as sum } from "./math.js";
```

3.2 现代操作符

3.2.1 可选链 (Optional Chaining `?.`)

安全访问深层嵌套属性，遇到 null/undefined 返回 undefined 而不报错。

```
const city = user?.address?.city; // 如果 user 或 address 不存在，返回 undefined
```

3.2.2 空值合并 (Nullish Coalescing `??`)

只有当左侧为 `null` 或 `undefined` 时才返回右侧（区别于 `||` 会对 0 或 false 生效）。

```
const count = 0;
const num1 = count || 10; // 10 (因为 0 是 falsy)
const num2 = count ?? 10; // 0 (因为 0 不是 null/undefined)
```

3.3 函数式编程范式 (Functional Programming)

JavaScript 拥有一等公民函数（First-class Function），天然支持函数式编程。

3.3.1 纯函数 (Pure Function)

- 相同的输入永远得到相同的输出。
- 无副作用（不修改外部状态，不进行 I/O 操作）。

3.3.2 高阶函数 (Higher-Order Function)

- 接收函数作为参数，或返回一个函数的函数。
- 常见示例：`map`，`filter`，`reduce`。

3.3.3 函数柯里化 (Currying)

将多参数函数转换成一系列单参数函数的技术。

```
// 普通函数
function add(a, b, c) {
```

```
  return a + b + c;
}

// 柯里化
function curriedAdd(a) {
  return function (b) {
    return function (c) {
      return a + b + c;
    };
  };
}
// 箭头函数简写
const curriedAddArrow = (a) => (b) => (c) => a + b + c;

console.log(curriedAdd(1)(2)(3)); // 6
```

3.4 元编程 (Metaprogramming)

ES6 引入了 `Proxy` 和 `Reflect`，赋予了开发者拦截并自定义语言基本操作的能力。

3.4.1 Proxy

用于定义基本操作的自定义行为（如属性查找、赋值、枚举、函数调用等）。

```
const target = {
  message: "Hello",
};

const handler = {
  get: function (target, prop, receiver) {
    if (prop === "message") {
      return "World"; // 拦截读取操作
    }
    return Reflect.get(...arguments);
  },
  set: function (target, prop, value) {
    if (prop === "age" && !Number.isInteger(value)) {
      throw new TypeError("Age must be an integer");
    }
    target[prop] = value;
    return true;
  },
};

const proxy = new Proxy(target, handler);
console.log(proxy.message); // World
// proxy.age = 'test'; // Throws TypeError
```

3.5 正则表达式进阶 (Regular Expressions)

正则表达式是处理文本的强大工具。

3.5.1 常用元字符

- `\d`: 数字, `\w`: 字母数字下划线, `\s`: 空白符
- `^`: 开始, `$`: 结束
- `*`: 0 次或多次, `+`: 1 次或多次, `?`: 0 次或 1 次

3.5.2 贪婪与懒惰匹配

- **贪婪模式** (默认): 尽可能多地匹配。
- **懒惰模式** (后跟 `?`): 尽可能少地匹配。

```
const str = "123456";
console.log(str.match(/\d+/)[0]); // "123456" (贪婪)
console.log(str.match(/\d+?/)[0]); // "1" (懒惰)
```

3.5.3 先行断言 (Lookahead)

- `(?=...)`: 正向先行断言。
- `(?!...)`: 负向先行断言。

```
// 匹配后面跟着 'bar' 的 'foo'
console.log(/foo(?=bar)/.test("foobar")); // true
console.log(/foo(?=bar)/.test("foobaz")); // false
```

3.6 错误处理最佳实践 (Error Handling)

3.6.1 自定义错误类型

继承 `Error` 类来创建特定领域的错误。

```
class ValidationError extends Error {
  constructor(message) {
    super(message);
    this.name = "ValidationError";
  }
}

function validateUser(user) {
  if (!user.name) {
    throw new ValidationError("Name is required");
  }
}
```

3.6.2 全局错误捕获

- **浏览器**: `window.onerror`, `window.onunhandledrejection`
- **Node.js**: `process.on('uncaughtException')`, `process.on('unhandledRejection')`

```
window.addEventListener("unhandledrejection", (event) => {
  console.error("Unhandled promise rejection:", event.reason);
});
```

---

4. 源码实现模拟 (Source Code Simulation)

掌握核心 API 的底层实现是进阶的关键。

4.1 手写 `new` 操作符

`new` 做了四件事：

1. 创建一个新对象。
2. 将新对象的原型指向构造函数的 prototype。
3. 执行构造函数，this 指向新对象。
4. 判断返回值，如果是对象则返回，否则返回新对象。

```
function myNew(Constructor, ...args) {
  // 1. 创建新对象，且原型指向构造函数的 prototype
  const obj = Object.create(Constructor.prototype);

  // 2. 执行构造函数，绑定 this
  const result = Constructor.apply(obj, args);

  // 3. 返回结果处理
  return typeof result === "object" && result !== null ? result : obj;
}
```

4.2 手写 `call`, `apply`, `bind`

这三个方法的核心是改变 `this` 指向。原理是将函数设置为对象的属性并调用。

```
Function.prototype.myCall = function (context, ...args) {
  // context 为 null/undefined 时默认指向 window/global
  context = context || globalThis;

  // 使用 Symbol 防止属性名冲突
  const fnSymbol = Symbol();

  // 将当前函数（this）赋值给 context 的属性
  context[fnSymbol] = this;

  // 执行函数
  const result = context[fnSymbol](...args);

  // 删除属性
  delete context[fnSymbol];

  return result;
};
```

4.3 手写简易 Promise

实现 Promise 的核心状态流转和 `then` 链式调用。

```
class MyPromise {
  constructor(executor) {
```

```
    this.state = "pending";
    this.value = undefined;
    this.reason = undefined;
    this.onFulfilledCallbacks = [];
    this.onRejectedCallbacks = [];

    const resolve = (value) => {
      if (this.state === "pending") {
        this.state = "fulfilled";
        this.value = value;
        this.onFulfilledCallbacks.forEach((fn) => fn());
      }
    };

    const reject = (reason) => {
      if (this.state === "pending") {
        this.state = "rejected";
        this.reason = reason;
        this.onRejectedCallbacks.forEach((fn) => fn());
      }
    };

    try {
      executor(resolve, reject);
    } catch (err) {
      reject(err);
    }
  }

  then(onFulfilled, onRejected) {
    // 值的穿透处理
    onFulfilled =
      typeof onFulfilled === "function" ? onFulfilled : (value) => value;
    onRejected =
      typeof onRejected === "function"
        ? onRejected
        : (err) => {
            throw err;
          };

    return new MyPromise((resolve, reject) => {
      if (this.state === "fulfilled") {
        setTimeout(() => {
          // 模拟微任务
          try {
            const x = onFulfilled(this.value);
            resolve(x); // 简化版，未处理 x 是 Promise 的情况
          } catch (e) {
            reject(e);
          }
        });
      }
      if (this.state === "rejected") {
        setTimeout(() => {
          try {
```

```
            const x = onRejected(this.reason);
            resolve(x);
          } catch (e) {
            reject(e);
          }
        });
      }
      if (this.state === "pending") {
        this.onFulfilledCallbacks.push(() => {
          setTimeout(() => {
            try {
              const x = onFulfilled(this.value);
              resolve(x);
            } catch (e) {
              reject(e);
            }
          });
        });
        this.onRejectedCallbacks.push(() => {
          setTimeout(() => {
            try {
              const x = onRejected(this.reason);
              resolve(x);
            } catch (e) {
              reject(e);
            }
          });
        });
      }
    });
  }
}
```

---

5. JavaScript 设计模式 (Design Patterns)

5.1 单例模式 (Singleton)

保证一个类仅有一个实例，并提供一个访问它的全局访问点。

```
class Singleton {
  constructor() {
    if (!Singleton.instance) {
      Singleton.instance = this;
    }
    return Singleton.instance;
  }
}
const s1 = new Singleton();
const s2 = new Singleton();
console.log(s1 === s2); // true
```

5.2 发布-订阅模式 (Publish-Subscribe)

Vue 的 EventBus 或 Node.js 的 EventEmitter 都是基于此模式。

```
class EventEmitter {
  constructor() {
    this.events = {};
  }

  on(type, handler) {
    if (!this.events[type]) {
      this.events[type] = [];
    }
    this.events[type].push(handler);
  }

  emit(type, ...args) {
    if (this.events[type]) {
      this.events[type].forEach((handler) => handler(...args));
    }
  }

  off(type, handler) {
    if (this.events[type]) {
      this.events[type] = this.events[type].filter((h) => h !== handler);
    }
  }
}
```

5.3 策略模式 (Strategy)

定义一系列的算法，把它们一个个封装起来，并且使它们可以相互替换。避免大量的 `if-else`。

```
const strategies = {
  S: (salary) => salary * 4,
  A: (salary) => salary * 3,
  B: (salary) => salary * 2,
};

const calculateBonus = (level, salary) => {
  return strategies[level](salary);
};

console.log(calculateBonus("S", 20000)); // 80000
```

---

6. 性能优化 (Performance Optimization)

6.1 防抖与节流 (Debounce & Throttle)

前端高频事件处理（如 resize, scroll, input）的必备优化手段。

- **防抖 (Debounce)**: N 秒内只执行一次，如果 N 秒内再次触发，重新计时。（应用：搜索框联想）
- **节流 (Throttle)**: N 秒内只执行一次，无论触发多少次。（应用：滚动加载、窗口缩放）

```
// 防抖
function debounce(fn, delay) {
  let timer = null;
  return function (...args) {
    if (timer) clearTimeout(timer);
    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

// 节流
function throttle(fn, delay) {
  let lastTime = 0;
  return function (...args) {
    const now = Date.now();
    if (now - lastTime > delay) {
      fn.apply(this, args);
      lastTime = now;
    }
  };
}
```

6.2 内存优化

- **解除引用**: 不再使用的对象设置为 `null`。
- **使用 WeakMap/WeakSet**: 键是弱引用，不影响 GC 回收。
- **避免长列表 DOM**: 使用虚拟滚动 (Virtual List)。

---

7. 适用场景 (Applicable Scenarios)

7.1 推荐场景

- **Web 前端交互**: DOM 操作、事件处理、AJAX 请求、动画效果。
- **单页应用 (SPA)**: 使用 React, Vue, Angular 构建复杂的富客户端应用。
- **服务端开发 (Node.js)**: 高并发 I/O 密集型应用，如 API Gateway、实时聊天、流媒体服务。
- **移动端开发**: React Native, Ionic 等跨平台框架。
- **桌面应用**: Electron (VS Code, Discord 就是用 Electron 开发的)。
- **Serverless**: AWS Lambda, Vercel Edge Functions 等轻量级函数计算。

7.2 不推荐场景

- **CPU 密集型计算**: 如深度学习训练、复杂的图像/视频处理（虽然 WebAssembly 正在改善这一点，但原生 JS 依然不擅长）。Node.js 的单线程模型在处理 CPU 密集任务时会阻塞 Event Loop。
- **对类型安全要求极高的底层系统**: 虽然 TypeScript 解决了大部分问题，但如果需要极致的内存控制和类型安全，Rust 或 C++ 是更好选择。

---

8. 行业对比 (Industry Comparison)

8.1 JavaScript vs TypeScript

维度	JavaScript	TypeScript
类型系统	动态弱类型	静态强类型（超集）
编译	解释执行/JIT	需编译为 JS
开发体验	灵活，但易出错	智能提示，代码健壮
适用规模	小型脚本、快速原型	中大型企业级项目

8.2 JavaScript (Node.js) vs Python

维度	Node.js	Python
并发模型	事件驱动，非阻塞 I/O	多线程（受 GIL 限制）/ 协程
运行速度	快 (V8 引擎)	较慢 (CPython)
应用领域	Web 全栈、实时服务	数据科学、AI、自动化脚本

---

9. 注意事项 (Precautions)

9.1 内存泄漏 (Memory Leaks)

虽然 JS 有垃圾回收（Mark-and-Sweep），但不良代码仍会导致内存泄漏：

- **意外的全局变量**: 忘记 `var/let/const`。
- **被遗忘的定时器**: `setInterval` 未清除。
- **闭包**: 不当持有大对象引用。
- **DOM 引用**: JS 中保留了已删除 DOM 节点的引用。

9.2 浮点数精度问题

JS 使用 IEEE 754 双精度浮点数，导致 `0.1 + 0.2 !== 0.3`。

- **解决方案**: 使用 `Number.EPSILON` 比较，或使用 `decimal.js` 库，或在金额计算时转为整数（分）。

9.3 异步陷阱

- **forEach 与 async**: `Array.prototype.forEach` 不支持 async/await，循环内的 await 不会阻塞后续代码。应使用 `for...of`。
- **Promise 异常吞没**: 忘记写 `.catch()` 或 `try-catch`，导致未捕获的 Promise Rejection。

---

10. 常见问题 (FAQ)

10.1 `var`, `let`, `const` 的具体区别是什么?

- **作用域**: `var` 是函数作用域，`let/const` 是块级作用域。
- **提升**: `var` 声明会被提升并初始化为 undefined；`let/const` 声明也会被提升但保留在 TDZ（暂时性死区），访问会报错。
- **重复声明**: `var` 允许，`let/const` 禁止。
- **全局对象属性**: 全局 `var` 会成为 window 属性，`let/const` 不会。

10.2 什么是事件委托（Event Delegation）?

利用事件冒泡机制，将子元素的事件监听器绑定到父元素上。

- **优点**: 减少内存占用（绑定更少的监听器）；动态添加的子元素也能响应事件。

10.3 `==` 和 `===` 有什么区别?

- `==` (宽松相等): 会进行类型转换。例如 `'1' == 1` 为 true。
- `===` (严格相等): 不转换类型，类型和值都必须相等。推荐始终使用 `===`。

10.4 如何判断一个变量是数组?

- `Array.isArray(val)` (推荐，ES5)
- `Object.prototype.toString.call(val) === '[object Array]'` (最通用)
- `val instanceof Array` (在多窗口/iframe 环境下可能失效)

10.5 深拷贝与浅拷贝的区别?

- **浅拷贝**: 只复制对象的引用或第一层属性（`Object.assign`, `...spread`）。
- **深拷贝**: 递归复制所有层级，新旧对象互不影响（`JSON.parse(JSON.stringify())` [有缺陷], `structuredClone()` [现代标准], `lodash.cloneDeep`）。

10.6 `this` 的指向规则是什么?

1. **默认绑定**: 独立函数调用，指向全局对象（严格模式下为 undefined）。
2. **隐式绑定**: `obj.method()`，指向 `obj`。
3. **显式绑定**: `call`, `apply`, `bind`，指向指定对象。
4. **new 绑定**: 指向新创建的实例。
5. **箭头函数**: 继承外层作用域的 `this`。

- 优先级: new > 显式 > 隐式 > 默认。

10.7 什么是 BFC (Block Formatting Context)?

虽然是 CSS 概念，但常在 JS 面试中问及。块级格式化上下文，是一个独立的渲染区域。

- **触发条件**: float 不为 none, overflow 不为 visible, position 为 absolute/fixed, display 为 inline-block/flex/grid 等。
- **作用**: 清除浮动、防止 margin 重叠、自适应两栏布局。

10.8 CommonJS 和 ES Modules 的循环引用处理有什么不同?

- **CommonJS**: 遇到循环引用时，只输出已经执行的部分，再次遇到直接返回缓存，可能导致获取不完整的数据。
- **ESM**: 利用动态引用（Binding），只要在使用时模块已经加载完毕即可正常访问，支持循环引用。

10.9 什么是高阶组件 (HOC)?

(React 相关但 JS 基础) 接收一个组件并返回一个新组件的函数。本质是设计模式中的装饰器模式。

10.10 如何实现一个 sleep 函数?

```
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
// 使用: await sleep(1000);
```

10.11 `null` 和 `undefined` 有什么区别?

- `undefined`: 变量声明了但未赋值，或对象属性不存在。代表“缺少值”。
- `null`: 赋值了，但值为空。代表“空对象指针”。
- **陷阱**: `typeof null === 'object'` (历史遗留 Bug)。
- **最佳实践**: 永远不要显式赋值 `undefined`，想表示空值请用 `null`。

10.12 为什么 `0.1 + 0.2 !== 0.3`?

这是 IEEE 754 双精度浮点数标准的特性，并非 JS 独有。
解决: `Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON` 或使用 `decimal.js` 库。

10.13 `map` 和 `forEach` 选哪个?

- `map`: 返回新数组，不改变原数组。用于数据转换。支持链式调用。
- `forEach`: 无返回值 (`undefined`)，用于执行副作用（如打印日志、写入数据库）。
- **原则**: 如果你需要结果，用 `map`；如果你只需要做事，用 `forEach`。

10.14 箭头函数能作为构造函数吗?

不能。箭头函数没有自己的 `this`，也没有 `prototype` 属性，因此不能使用 `new` 关键字，否则会报错 `TypeError: ... is not a constructor`。

---

11. 进阶话题：V8 引擎与性能优化

深入理解 JS 引擎（V8）的工作原理，有助于写出高性能的代码。

11.1 JIT 编译 (Just-In-Time)

JS 不再是单纯的解释型语言。V8 引擎包含两个核心组件：

1. **Ignition (解释器)**: 将 JS 源码转换为字节码 (Bytecode) 并执行。
2. **TurboFan (编译器)**: 将热点代码 (Hot Code) 编译为机器码 (Machine Code) 以提高执行效率。

优化策略: 保持代码的“稳定性”。如果一个函数接收的参数类型总是相同的，TurboFan 就能生成高效的机器码；如果参数类型频繁变化，代码会被“去优化” (Deoptimization)，回退到解释器执行。

11.2 隐藏类 (Hidden Classes / Shapes)

JS 是动态语言，对象属性可以随时添加。为了加速属性访问，V8 使用“隐藏类”。

```
function Point(x, y) {
  this.x = x;
  this.y = y;
}
const p1 = new Point(1, 2);
const p2 = new Point(3, 4);
// p1 和 p2 共享同一个隐藏类 (Shape)

p1.z = 5; // p1 的隐藏类改变了！
// 现在 p1 和 p2 拥有不同的隐藏类，v8 无法对它们进行统一优化。
```

最佳实践:

- 在构造函数中一次性初始化所有属性。
- 尽量按照相同的顺序初始化对象属性。
- 避免使用 `delete` 删除属性（这会破坏隐藏类），建议赋值为 `null`。

11.3 垃圾回收 (Garbage Collection)

V8 的垃圾回收器名为 Orinoco，基于分代回收算法。

- **新生代 (New Space)**: 存放生命周期短的对象（如临时变量）。使用 **Scavenge 算法**（Cheney 算法），将空间分为 From 和 To 两块，存活对象在两者间复制。速度极快。
- **老生代 (Old Space)**: 存放生命周期长或常驻内存的对象。使用 **Mark-Sweep-Compact (标记-清除-整理)** 算法。
  - **标记**: 遍历引用链，标记活动对象。
  - **清除**: 回收未标记对象的内存。
  - **整理**: 移动对象，减少内存碎片（成本较高，非必须不执行）。

优化建议:

- 避免全局变量造成的内存泄漏。
- 及时解绑 DOM 事件。
- 慎用闭包，避免无意中持有大对象的引用。

---

12. ECMAScript 2024/2025 新特性前瞻

标准委员会 (TC39) 每年都在推动 JS 进化。

12.1 Object.groupBy (ES2024)

原生支持数组分组，不再需要 Lodash。

```
const inventory = [
  { name: "asparagus", type: "vegetables" },
  { name: "bananas", type: "fruit" },
  { name: "goat", type: "meat" },
];

const result = Object.groupBy(inventory, ({ type }) => type);
/*
{
  vegetables: [{ name: "asparagus", ... }],
  fruit: [{ name: "bananas", ... }],
  meat: [{ name: "goat", ... }]
}
*/
```

12.2 Promise.withResolvers (ES2024)

简化 Promise 的创建，特别是需要将 resolve/reject 暴露到外部时。

```
// 以前
let resolve, reject;
const promise = new Promise((res, rej) => {
  resolve = res;
  reject = rej;
});

// 现在
const { promise, resolve, reject } = Promise.withResolvers();
```

12.3 Temporal API (即将到来)

旨在彻底解决 `Date` 对象的痛点。

```
// 以前的 Date 操作很痛苦
// 未来的 Temporal
// const now = Temporal.Now.plainDateTimeISO();
// const future = now.add({ days: 7 });
```

---

13. JavaScript 执行机制深度剖析 (Deep Dive: Execution Mechanism)

深入理解 JavaScript 的执行机制是成为高级开发者的必经之路。

13.1 V8 引擎工作流程

V8 是 Google 开发的开源高性能 JavaScript 和 WebAssembly 引擎。它的核心职责是将 JavaScript 代码编译成机器码并执行。

1. 解析 (Parsing)

- **Scanner**: 将源代码分解成 tokens（词法分析）。
- **Parser**: 根据语法规则，将 tokens 转换成抽象语法树（AST）。

2. 解释与编译 (Ignition & TurboFan)

- **Ignition (解释器)**: 将 AST 转换成字节码（Bytecode）。字节码比机器码更紧凑，能减少内存占用。Ignition 直接执行字节码。
- **TurboFan (优化编译器)**: 在代码运行过程中，Ignition 会收集类型信息（Profiling）。如果某段代码执行频率很高（Hot Code），TurboFan 会利用这些信息将其编译成高度优化的机器码。

3. 优化与去优化 (Optimization & Deoptimization)

- **优化**: 假设变量类型不变，直接生成特定类型的机器码。
- **去优化**: 如果变量类型突然改变（例如数组从全整数变成了混合类型），优化的机器码失效，V8 会回退（Deoptimize）到字节码执行。

> **性能启示**: 保持对象结构稳定，避免动态添加属性；保持数组类型一致，能显著提高 V8 的执行效率。

13.2 执行上下文 (Execution Context)

每次代码运行时，都会创建一个执行上下文。

- **全局执行上下文**: 默认的上下文，`this` 指向 window (浏览器) 或 global (Node.js)。
- **函数执行上下文**: 每次调用函数时创建。
- **Eval 执行上下文**: 不推荐使用。

执行上下文生命周期:

1. **创建阶段**:
    - 创建变量对象 (VO) / 活动对象 (AO)。
    - 建立作用域链 (Scope Chain)。
    - 确定 `this` 指向。
2. **执行阶段**:
    - 变量赋值。
    - 函数引用。
    - 执行代码。

13.3 调用栈 (Call Stack)

JavaScript 是单线程的，使用调用栈来管理函数的执行顺序。栈遵循 LIFO (后进先出) 原则。

```
function a() {
  b();
}
function b() {
  c();
}
function c() {
  console.log("Hello");
}
a();
```

栈操作:

1. `main()` 入栈。
2. `a()` 入栈。
3. `b()` 入栈。
4. `c()` 入栈。
5. `console.log` 入栈并执行，出栈。
6. `c()` 出栈 -> `b()` 出栈 -> `a()` 出栈。

如果递归调用没有终止条件，会导致**栈溢出 (Stack Overflow)**。

---

14. 高级异步编程模式 (Advanced Asynchronous Patterns)

14.1 事件循环模型 (Event Loop Model)

JavaScript 的并发模型基于“事件循环”。

宏任务 (Macrotasks) 与 微任务 (Microtasks)

- **Macrotasks**: `setTimeout`, `setInterval`, `setImmediate` (Node.js), I/O, UI Rendering.
- **Microtasks**: `Promise.then`, `process.nextTick` (Node.js), `MutationObserver`.

循环过程:

1. 执行同步代码（调用栈清空）。
2. 执行当前所有的**微任务**（清空微任务队列）。
3. 执行**一个**宏任务。
4. 渲染 UI (浏览器)。
5. 回到步骤 2。

> **关键点**: 微任务优先级高于宏任务。在进入下一个宏任务之前，必须清空微任务队列。

示例分析

```
console.log("1"); // 同步

setTimeout(() => {
  console.log("2"); // 宏任务
}, 0);

Promise.resolve().then(() => {
  console.log("3"); // 微任务
});

console.log("4"); // 同步

// 输出顺序: 1 -> 4 -> 3 -> 2
```

14.2 并发控制 (Concurrency Control)

在处理大量并发请求时，需要控制并发数量以避免服务器过载。

```
async function asyncPool(poolLimit, array, iteratorFn) {
  const ret = []; // 存储所有的异步任务
  const executing = []; // 存储正在执行的异步任务

  for (const item of array) {
    const p = Promise.resolve().then(() => iteratorFn(item, array));
    ret.push(p); // 保存新的异步任务

    if (poolLimit <= array.length) {
      // 当任务完成后，从 executing 数组中移除
      const e = p.then(() => executing.splice(executing.indexOf(e), 1));
      executing.push(e); // 保存正在执行的异步任务

      if (executing.length >= poolLimit) {
        // 等待较快的任务执行完成
        await Promise.race(executing);
      }
    }
  }
  return Promise.all(ret);
}
```

---

15. 函数式编程与设计模式 (FP & Design Patterns)

15.1 函数式编程核心概念

JavaScript 支持函数式编程，这在 React 等框架中尤为重要。

1. **纯函数 (Pure Functions)**:
    - 相同的输入永远得到相同的输出。
    - 无副作用（不修改外部状态，不打印日志，不发网络请求）。
    - 易于测试和缓存。
2. **柯里化 (Currying)**:
    - 将接受多个参数的函数变换成接受一个单一参数的函数，并且返回接受余下参数的新函数。

```
const add = (a) => (b) => a + b;
const add5 = add(5);
console.log(add5(2)); // 7
```

3. **函数组合 (Composition)**:
   - 将两个或多个函数组合成一个新函数。

```
const compose = (f, g) => (x) => f(g(x));
```

15.2 常见设计模式

1. **单例模式 (Singleton)**: 保证一个类仅有一个实例。

```
class Singleton {
  static instance;
  constructor() {
    if (Singleton.instance) return Singleton.instance;
    Singleton.instance = this;
  }
}
```

2. **观察者模式 (Observer)**: 定义对象间的一对多依赖，当一个对象状态改变时，所有依赖它的对象都得到通知。
   - 前端应用：DOM 事件监听，Vue 的响应式系统。
3. **工厂模式 (Factory)**: 创建对象的接口，让子类决定实例化哪一个类。
4. **代理模式 (Proxy)**: 为其他对象提供一种代理以控制对这个对象的访问。
   - ES6 `Proxy` 对象是实现此模式的利器。

---

16. 性能优化策略 (Performance Optimization)

16.1 内存泄漏防范

1. **意外的全局变量**:

```
function foo() {
  bar = "this is global"; // 忘记使用 var/let/const
}
```

2. **被遗忘的计时器**: `setInterval` 如果不清除，其引用的外部变量无法被 GC。
3. **闭包**: 不当使用闭包会导致作用域链上的变量无法释放。
4. **DOM 引用**: JS 中保留了 DOM 节点的引用，即使节点从页面移除，内存中依然存在。

16.2 代码层面优化

1. **使用局部变量**: 查找局部变量比全局变量快（作用域链查找）。
2. **避免强制同步布局 (Layout Thrashing)**: 避免在循环中读取并写入 DOM 属性。
3. **使用位运算**: 在特定场景下（如权限控制）比常规运算快。
4. **对象属性访问**: 保持对象形状（Shape）稳定，利用 V8 的隐藏类（Hidden Classes）机制。

17. 适用场景 (Applicable Scenarios)

场景	说明	示例
前端交互	DOM 操作、事件处理、动画	React, Vue, jQuery
服务端开发	高并发 I/O 密集型应用	Node.js, Next.js API Routes
移动端开发	跨平台移动应用	React Native, Ionic
桌面应用	跨平台桌面应用	Electron (VS Code, Slack)
全栈开发	前后端同构，共享逻辑	Next.js (SSR + Hydration)

14. 注意事项 (Precautions)

1. **全局变量污染**: 尽量避免定义全局变量，使用 `const` / `let` 和模块化 (`import` / `export`) 来隔离作用域。
2. **内存泄漏**:
   - 及时清除定时器 (`clearInterval`).
   - 解绑不再需要的事件监听器.
   - 避免不必要的闭包持有大对象引用.
3. **安全性 (XSS)**: 永远不要信任用户输入。避免使用 `innerHTML` 渲染用户内容，除非经过严格清洗 (DOMPurify)。
4. **原型污染**: 避免直接修改 `Object.prototype`。使用 `Object.create(null)` 创建无原型对象作为字典。
5. **类型转换陷阱**: 尽量使用 `===` (全等) 代替 `==`，避免隐式类型转换带来的不可预知结果（如 `[] == ![]` 为 true）。

15. 总结 (Summary)

15.1 核心要点回顾

JavaScript 从简单的脚本语言演变为如今的全栈霸主，其核心在于灵活的**对象模型（原型链）**、高效的**异步机制（事件循环）**以及不断进化的**标准规范（ECMAScript）**。掌握闭包、原型、Promise 和 ES6+ 新特性是成为高级 JS 开发者的必经之路。深入理解 V8 引擎的工作原理和内存管理，能够帮助开发者编写出更高性能、更健壮的代码。

15.2 学习建议

- **打好基础**: 不要跳过原型链、this 指向、事件循环等“难点”，它们是理解框架原理的基石。
- **拥抱现代 JS**: 熟练使用 `map/filter/reduce`、解构、Async/Await，写出声明式、简洁的代码。
- **关注运行时**: 理解 V8 引擎的工作原理（调用栈、堆、GC）能帮你写出性能更好的代码。
- **实践出真知**: 多写代码，多 Debug，理解代码在内存中的实际表现。手写核心 API（如 Promise, new, call）是检验掌握程度的最好方式。

> 关键版本记录：

> - ES6 (2015): let/const, class, module, promise
> - ES2017: async/await
> - ES2020: BigInt, Optional Chaining, Nullish Coalescing
> - ES2024: Object.groupBy, Promise.withResolvers

闲鱼号: xy769003723321
店铺名称: 高质量 IT 资源铺
个人整理 禁止倒卖